GII-ADE-M. Relación de problemas. Tema 2. 15/10/2026

Indicaciones generales. Resolver justificando los invariantes de seguridad y las condiciones de progreso. Salvo indicación contraria, las operaciones sobre semáforos son atómicas, los procesos pueden ejecutarse a velocidades arbitrarias y no se supone un orden FIFO de desbloqueo. Distinguir entre ausencia de interbloqueo y ausencia de inanición. En los problemas con monitores, indicar la semántica utilizada (Hoare/SU) y emplear `while` para revalidar condiciones cuando corresponda.

I. Semáforos

37. Tres procesos ejecutan indefinidamente las siguientes secuencias:

```
process P1: while true do begin a; b; c end
process P2: while true do begin d; e; f end
process P3: while true do begin g; h; i end
```

Resolver *independientemente* mediante semáforos los siguientes requisitos. En (a)–(c), la condición se refiere a la *misma ronda* k de ejecución de cada proceso, numerada desde 1; evitar que señales de rondas futuras satisfagan esperas de rondas anteriores. Si una solución exige una sincronización adicional de fin de ronda, explicitarla.

(a) P_2 ejecuta e_k solamente después de que haya terminado a_k *o* g_k (basta una de las dos).

(b) P_2 ejecuta e_k solamente después de que hayan terminado a_k *y* g_k.

(c) P_2 ejecuta e_k y P_3 ejecuta h_k solamente después de que P_1 haya terminado b_k; ambas autorizaciones son independientes.

(d) A lo sumo dos de las operaciones b, f y h pueden estar ejecutándose simultáneamente. Suponer que cada operación tiene un inicio y un fin distinguibles.

38. Dos procesos comparten exclusivamente la variable global `x`. Las constantes `n` y `c` son inmutables; `m` y `d` son locales:

```
process P1:                        process P2:
  while true do begin               while true do begin
    m := 2*x - n;                      d := leer_teclado();
    imprimir(m);                       x := d - 5*c;
  end                                 end
```

Se considera *producido* un valor cuando P_2 termina la asignación a `x`, y *consumido* cuando P_1 termina de leerlo. Diseñar dos protocolos distintos mediante semáforos:

(a) Consumir exactamente una vez todos los valores producidos, en orden y sin sobrescrituras.

(b) Consumir exactamente los valores de orden impar (1, 3, 5, ...), descartando los de orden par, sin duplicaciones ni lecturas inconsistentes.

Para ambos casos indicar valores iniciales de semáforos, ubicación de `wait/post` y si el productor puede bloquearse. No modificar la fórmula de cálculo de `m`.

39. Un aseo compartido admite simultáneamente *o bien* una mujer *o bien* hasta cinco hombres; nunca ambos grupos a la vez. Resolver mediante semáforos POSIX *no nombrados*, para hilos del mismo proceso, dos variantes:

(a) Prioridad estricta de hombres: si hay hombres esperando, ninguna mujer nueva puede entrar, incluso si el aforo masculino está completo.

(b) Prioridad estricta de mujeres: si hay mujeres esperando, ningún hombre nuevo puede entrar.

Diseñar las operaciones de entrada y salida de cada grupo, con contadores protegidos y señales de condición implementadas con semáforos. Evitar esperas activas y despertar incorrectamente a varios hilos para una sola plaza. Discutir si la prioridad estricta puede causar inanición y por qué la semántica POSIX no garantiza FIFO.

II. Monitores

40. Un monitor garantiza exclusión mutua al ejecutar sus procedimientos. Explicar qué significa que un procedimiento sea *reentrante*, por qué exclusión mutua y reentrancia no son propiedades equivalentes y en qué situaciones (por ejemplo, llamadas recursivas, estado externo o invocaciones concurrentes) es necesario exigir reentrancia. Distinguir un monitor no recursivo de uno recursivo.

41. Un sistema dispone de $N_1 > 0$ unidades indistinguibles de recurso tipo 1 y $N_2 > 0$ de tipo 2. Diseñar un monitor con semántica de señalización urgente (SU) que exporte `solicitar(tipo)` y `liberar(tipo)`, con *tipo* $\in \{1, 2\}$, y garantice que nunca se conceden más de N_t unidades de cada tipo.

(a) Implementar ambos procedimientos para solicitudes individuales.

(b) Mostrar una secuencia de solicitudes de dos o más procesos que pueda producir interbloqueo si cada proceso necesita conservar una unidad de cada tipo simultáneamente. Precisar las hipótesis sobre capacidades y adquisiciones.

(c) Extender la interfaz con `solicitar(0)`, que reserva *atómicamente* una unidad de cada tipo, y `liberar(0)`, que devuelve ambas. Las peticiones dobles pendientes tendrán prioridad sobre las individuales *cuando ambas unidades estén disponibles*; no bloquear innecesariamente recursos utilizables cuando una petición doble aún no pueda satisfacerse. Justificar seguridad y ausencia de interbloqueo por adquisición parcial; discutir inanición.

42. Resolver el problema de lectores y escritores mediante monitores. Se permiten varios lectores simultáneos o un único escritor, nunca ambos. Presentar tres políticas *independientes*:

(a) Prioridad de lectores.

(b) Prioridad de escritores.

(c) Orden de llegada FIFO por solicitudes, admitiendo como lote a lectores consecutivos situados al principio de la cola y sin adelantar a un escritor anterior.

Definir `entrarLeer`, `salirLeer`, `entrarEscribir` y `salirEscribir`. Para (c), utilizar tickets o una cola explícita de solicitudes; no suponer que las variables condición son FIFO. Analizar la posible inanición en cada política.

43. Vehículos procedentes del norte y del sur atraviesan un puente de un solo carril. Pueden circular simultáneamente varios coches en la *misma* dirección, pero nunca en sentidos opuestos.

(a) Completar un monitor con procedimientos `entrarNorte`, `salirNorte`, `entrarSur` y `salirSur`. La política inicial permite entrar si no hay vehículos del sentido contrario sobre el puente. Señalar expresamente si esa política garantiza o no ausencia de inanición.

(b) Modificarlo para que, cuando haya demanda en ambos sentidos, se concedan turnos de *hasta* diez entradas consecutivas por sentido; terminado el cupo, no se permiten nuevas

entradas de ese sentido hasta que el puente quede vacío y se transfiera el turno. Si solo hay demanda de un sentido, permitir circulación sin interrupciones innecesarias. Explicar cómo se contabilizan las entradas y cómo se evita que un flujo continuo impida vaciar el puente.

44. Un comedor tiene una olla de capacidad $M > 0$ raciones y $N > 0$ comensales. Cada comensal retira una ración y come fuera del monitor. Si la olla está vacía, el primer comensal que lo detecta solicita una reposición al cocinero; el resto espera sin generar nuevas solicitudes. El cocinero duerme hasta recibir una petición y repone exactamente M raciones.

```
process Comensal[i:1..N]:
  while true do begin Olla.servir(); comer() end
process Cocinero:
  while true do begin Olla.esperarPeticion();
                      Olla.reponer() end
```

Implementar el monitor `Olla` con semántica SU, evitando peticiones duplicadas, pérdida de avisos, interbloqueos y consumo de raciones inexistentes. Especificar la cantidad inicial de raciones y el estado inicial del cocinero.

45. Una cuenta bancaria compartida tiene saldo inicial $S_0 \geq 0$. El monitor exporta `ingresar(c)` y `retirar(c)`, para $c > 0$. Nunca puede quedar saldo negativo. Resolver las siguientes políticas:

(a) Una retirada se concede en cuanto hay saldo suficiente, aunque otras retiradas anteriores estén esperando. Proponer (a.1) una implementación con colas de condición ordinarias, sin garantías de prioridad, y (a.2) otra con una estructura de espera ordenada por importe solicitado (menor importe primero), indicando cómo despertar únicamente peticiones elegibles.

(b) Orden estricto de llegada: si la primera retirada pendiente no puede satisfacerse, ninguna posterior puede adelantarla, aunque su importe sea menor. Proponer (b.1) una cola FIFO explícita y (b.2) una cola de condición con prioridad por número de ticket, si el lenguaje la proporciona.

Utilizar semántica SU. Justificar que las estructuras propuestas preservan el saldo y aclarar las hipótesis de progreso: sin ingresos futuros suficientes, ninguna política puede garantizar la finalización de toda retirada.

46. $n \geq 1$ procesos comparten un recurso indivisible R. Cada proceso P_i tiene prioridad fija i, y un índice menor implica mayor prioridad. Implementar un monitor con `solicitar(i)` y `liberar(i)` tal que solo un proceso posea R y, al liberarlo, se seleccione la solicitud pendiente de mayor prioridad. Si hay varias solicitudes del mismo índice, definir un criterio de desempate. Distinguir prioridad en la *adjudicación* del recurso de expulsión del poseedor actual. Analizar inanición y proponer una modificación con envejecimiento de prioridades.

47. Una barrera cíclica de dos participantes ofrece `llegar()`: el primero espera al segundo y ambos pueden continuar cuando han llegado dos participantes. Implementarla exclusivamente con semáforos y contadores protegidos. La barrera se reutiliza indefinidamente y pueden intervenir hilos distintos en cada ronda. Evitar que una señal de la ronda k libere erróneamente a un hilo de la ronda $k + 1$. Justificar la reutilización correcta mediante dos torniquetes o un mecanismo equivalente de generaciones.

48. Cinco filósofos $F_0, \ldots, F_4$ se sientan alrededor de una mesa y comparten cinco tenedores $T_0, \ldots, T_4$. El filósofo F_i necesita simultáneamente T_i y $T_{(i+1) \bmod 5}$ para comer. La interfaz original ofrece `tomar(tenedor,filosofo)` y `soltar(tenedor)`.

(a) Diseñar un monitor que implemente la adquisición individual de tenedores y su liberación. Corregir explícitamente el código del filósofo para que libere ambos tenedores tras comer.

(b) Mostrar una ejecución que produzca interbloqueo cuando todos adquieren primero T_i y después $T_{(i+1) \bmod 5}$.

(c) Diseñar una solución que limite a cuatro el número de filósofos que pueden comenzar a adquirir tenedores simultáneamente. Indicar cómo se libera el permiso y demostrar ausencia de interbloqueo. ¿Garantiza también ausencia de inanición?

49. Diseñar un canal de capacidad $K \geq 1$ con $p \geq 2$ productores y $q \geq 2$ consumidores. Cada productor envía una secuencia finita de elementos y anuncia su terminación; los consumidores reciben elementos hasta que el canal está cerrado y vacío. Exigir: ausencia de pérdidas y duplicados, orden FIFO global de inserción, bloqueo de productores con búfer lleno, bloqueo de consumidores con búfer vacío mientras queden productores, y terminación de todos los consumidores sin insertar marcadores artificiales en el búfer. Implementar con semáforos y justificar la ausencia de señales perdidas y la gestión de la última terminación. Discutir qué cambia si se cancela un productor bloqueado.

50. Hay tres clases de recursos con capacidades (N_A, N_B, N_C) y procesos que solicitan vectores (a, b, c) con $0 \leq a \leq N_A$, $0 \leq b \leq N_B$, $0 \leq c \leq N_C$. Diseñar un monitor que conceda cada vector de forma atómica (todo o nada), permita liberar vectores parciales de una reserva propia y evite interbloqueo por retención y espera. Comparar dos políticas de admisión: FIFO estricto y selección de la primera petición satisfacible. Construir un escenario concreto donde la segunda mejore utilización pero produzca inanición; proponer una política de envejecimiento y especificar qué propiedad de progreso puede garantizarse bajo hipótesis explícitas sobre liberaciones.

51. Diseñar una barrera reutilizable para $N \geq 3$ hilos con generaciones numeradas. Antes de llegar a una barrera, un hilo puede darse de baja para *rondas futuras*; además, una espera puede terminar por plazo vencido. Definir rigurosamente cuándo queda fijado el conjunto de participantes de una generación, cómo se contabilizan las bajas y qué ocurre si vence el plazo de uno de ellos: elegir y justificar una semántica de *barrera rota* que despierte a todos los participantes de la misma generación. Implementar el protocolo con monitor y variables de condición, sin mezclar señales entre generaciones. Identificar invariantes y posibles carreras entre la última llegada y el vencimiento del plazo.

52. Extender el monitor del problema 43 para incorporar ambulancias en ambos sentidos. Una ambulancia tiene prioridad sobre coches ordinarios *en espera*, pero no puede expulsar vehículos que ya están cruzando. Mantener seguridad direccional, cupos máximos de diez entradas ordinarias cuando ambos sentidos esperan y evitar inanición de coches ordinarios aun si siguen llegando ambulancias. Formular una política concreta de alternancia/cupos que resuelva conflictos entre prioridad y justicia; diseñar el monitor, exponer invariantes y analizar un escenario adversario con llegadas continuas en ambos sentidos.

53. Dos servicios concurrentes A y B mantienen sendos saldos no negativos y permiten transferencias de importe $c > 0$ en ambas direcciones. Cada servicio dispone de su propio monitor; no se permite un monitor global ni mantener bloqueado un monitor mientras se espera en otro. Diseñar un protocolo de transferencia que garantice conservación del saldo total, ausencia de doble cargo y ausencia de transferencias parcialmente confirmadas, incluso si dos transferencias opuestas se ejecutan concurrentemente. En una primera parte, suponer ausencia de fallos y justificar interbloqueo y progreso. En una segunda, permitir la caída y recuperación de un servicio: especificar qué información persistente, identificadores únicos y estados de preparación/confirmación son necesarios, y explicar por qué no se puede garantizar terminación incondicional bajo fallos permanentes.